

SIMATS ENGINEERING

Department of Computer Science & Engineering ASSESSMENT TOOL 1-CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5	Assessment:	ASSESSMENT TOOL1-Constraint-Based Problem Solving
Weightage:	60%	Total Marks:	25
CO5Statement:	<i>Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.</i>		
Student Name:	Althaf A	Reg.No.:	192521257
Department:	Btech IT	Date:	31/8/2026

ANNEXURE A

Constraint-Based Problem Solving

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s

- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I G Devendra Nath Reddy certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Problem Understanding(25%)	Constraint Analysis(25%)	Solution Development (25%)	Application of Concepts(15%)	Justification(10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

ANSWERS

1. Real-Time Embedded System I/O Communication Mechanism

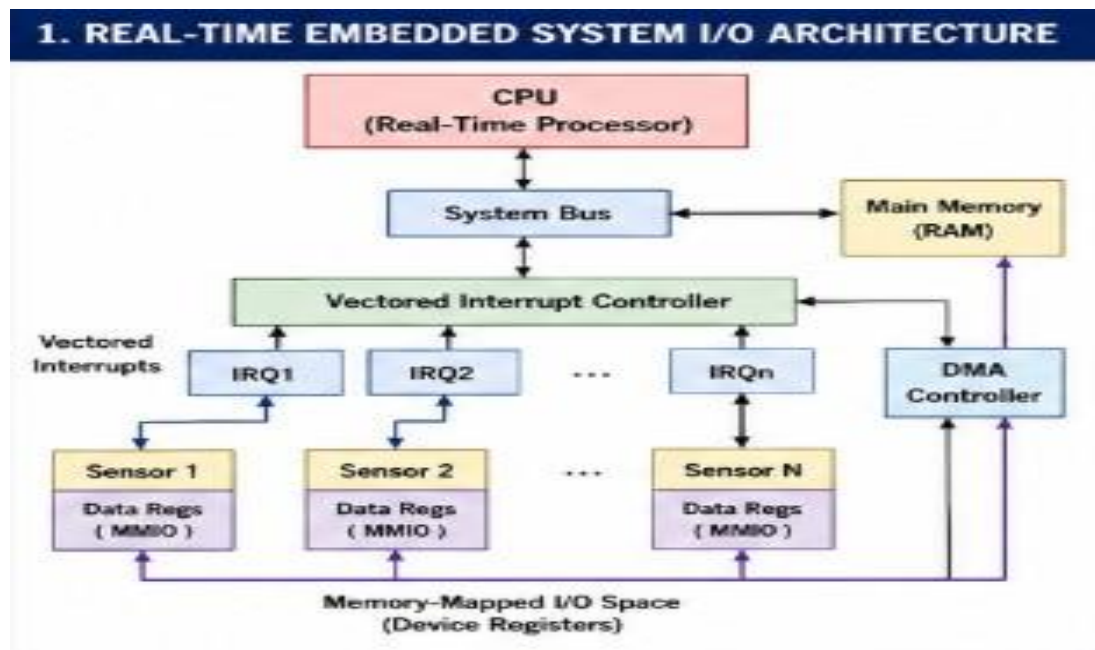
Given Constraints

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped I/O and I/O-mapped I/O for optimal latency

Proposed Design

A suitable design is a memory-mapped I/O subsystem with a vectored interrupt controller and DMA support.

Architecture



Working

Each sensor is connected to the interrupt controller. Whenever a sensor produces new data, it generates an interrupt.

Since the system uses vectored interrupts, every interrupt source has a predefined interrupt vector. The CPU can directly jump to the appropriate Interrupt Service Routine (ISR) without checking every device.

For example:

Sensor 1 → IRQ1 → Vector 1 → ISR1

Sensor 2 → IRQ2 → Vector 2 → ISR2

Sensor 3 → IRQ3 → Vector 3 → ISR3

This significantly reduces interrupt handling time.

Why Memory-Mapped I/O?

In memory-mapped I/O, device registers are assigned addresses in the processor's memory address space.

For example:

0x40000000 → Sensor 1 Status Register

0x40000004 → Sensor 1 Data Register

0x40000008 → Sensor 2 Status Register

The CPU can access these registers using normal load/store instructions.

Memory-mapped I/O is selected because it generally provides simpler and faster access on modern embedded processors and integrates naturally with the processor's memory system.

Reducing CPU Utilization

Sensors generate interrupts every 1 ms, which can create a very high interrupt rate. Therefore, the ISR should be extremely short.

The ISR should:

1. Identify the sensor using the interrupt vector.
2. Read only the required status/data.
3. Store data into a buffer.
4. Clear the interrupt.
5. Return immediately.

Heavy processing should be performed outside the ISR.

A **DMA controller** can also transfer sensor data directly into memory.

CPU Utilization

CPU utilization can approximately be represented as:

$$U = \sum_{i=1}^n f_i T_i$$

Where:

- f_i = interrupt frequency
- T_i = CPU time required per interrupt

For example, if 8 sensors generate 1000 interrupts/s each and each ISR takes 10 μ s:

$$U = 8 \times 1000 \times 10 \mu\text{s} = 0.08 = 8\%$$

Thus:

$$\text{CPU Utilization} = 8\% < 15\%$$

Advantages

- Very low interrupt dispatch latency
- Direct ISR selection
- Suitable for real-time applications
- Low CPU overhead
- Memory-mapped registers simplify programming
- DMA can further reduce CPU involvement

Final Design

Memory – Mapped I/O + Vectored Interrupts + DMA + Short ISRs

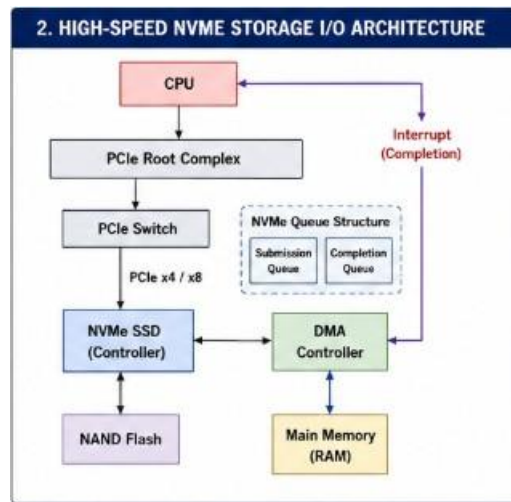
This architecture satisfies the **15% CPU utilization requirement** while providing fast real-time sensor handling.

2. High-Speed NVMe Storage I/O Subsystem

Given Constraints

- Storage throughput must be greater than 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- DMA controller must be integrated with interrupt notification

Proposed Architecture



NVMe Completion → Interrupt → CPU

Why NVMe over PCIe?

NVMe is specifically designed for high-speed non-volatile storage and communicates through PCIe.

A PCIe Gen3 x4 interface can provide approximately:

$$4 \times 985 \text{ MB/s} \approx 3.94 \text{ GB/s}$$

Therefore, PCIe x4 can satisfy a throughput requirement above 3 GB/s under suitable conditions.

Newer PCIe generations provide substantially higher bandwidth.

DMA-Based Data Transfer

The CPU should **not copy every block of data**.

Instead:

NVMe SSD



PCIe



DMA Controller



Main Memory

The CPU only prepares the transfer.

DMA Operation

1. CPU creates an I/O request.
2. NVMe command is placed into a submission queue.
3. NVMe controller processes the command.
4. Data is transferred through PCIe.
5. DMA transfers the data directly to RAM.
6. NVMe controller writes completion information.
7. Interrupt is generated.
8. CPU handles the completion event.

Example

Suppose a 1 GB block is transferred.

Without DMA:

SSD → CPU → Memory

The CPU must participate in the bulk transfer.

With DMA:

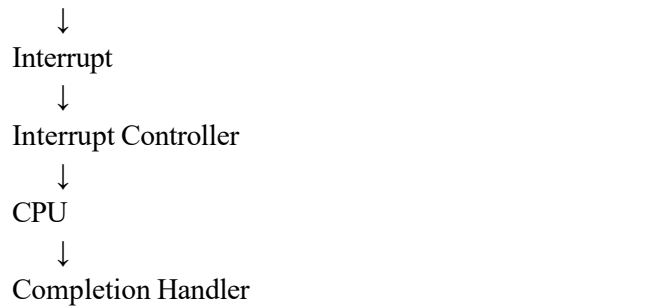
SSD → PCIe → DMA → Memory

The CPU only manages the control operations.

Interrupt Notification

After DMA completes:

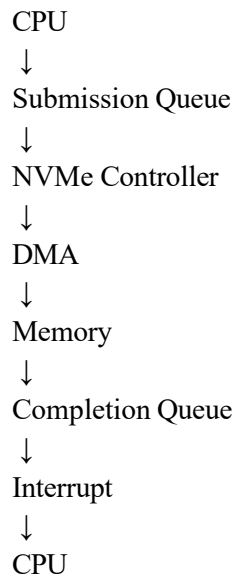
DMA Completion



The CPU does not need to continuously poll the device.

Queue-Based NVMe Design

NVMe supports submission and completion queues:



This reduces overhead and allows multiple outstanding I/O requests.

Throughput

Required:

$$Throughput > 3\text{ GB/s}$$

A PCIe x4 interface capable of approximately 3.94 GB/s raw payload-class bandwidth at PCIe Gen3 is theoretically sufficient, while **PCIe Gen4 x4** provides much more headroom.

Final Design

$$NVMe + PCIe\ x4\ or\ higher + DMA + Queue - based\ I/O + Interrupts$$

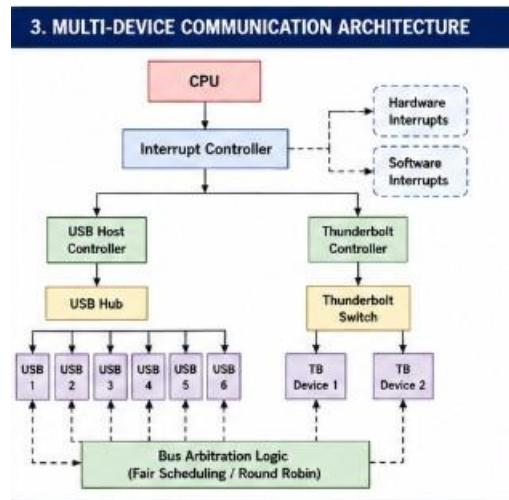
This provides high throughput while keeping CPU intervention minimal.

3. Multi-Device USB and Thunderbolt Communication Architecture

Given Constraints

- 6 USB peripherals
- 2 high-speed Thunderbolt devices
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

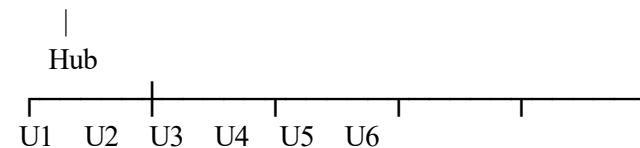
Proposed Architecture



USB Devices

Six USB devices are connected through a USB host controller/hub.

USB Host Controller



Thunderbolt Devices

Two high-speed devices are connected through the Thunderbolt controller.

Thunderbolt is given a high-performance communication path because these devices require greater bandwidth.

Hardware Interrupts

Hardware devices generate interrupts when:

- Data arrives
- Transfer completes
- Error occurs
- Device requires service

Example:

USB Device



Hardware IRQ



Interrupt Controller



USB ISR

Software Interrupts

Software interrupts can be used for lower-priority tasks such as:

- Deferred processing
- Driver-level task scheduling
- Protocol processing
- Background communication

Instead of doing extensive processing inside the hardware ISR:

Hardware IRQ



Short ISR



Software Interrupt



Deferred Processing

This prevents long ISR execution.

Avoiding Interrupt Starvation

Interrupt starvation occurs when low-priority devices do not receive CPU service because high-priority interrupts continuously occur.

A priority mechanism can be used:

Priority 1 → Critical Thunderbolt

Priority 2 → USB Controller

Priority 3 → USB Devices

Priority 4 → Background Software Interrupts

However, priority alone is not sufficient.

A **fairness mechanism** should be used so lower-priority devices eventually receive service.

Possible mechanisms:

- Round-robin arbitration
- Time-sliced servicing
- Priority aging
- Interrupt masking for short periods
- Bounded ISR execution time

Fair Bus Arbitration

A round-robin scheme can be used:

USB → TB → USB → TB → USB → TB

Each device/controller receives an opportunity to access the bus.

For example:

$$T_{USB1} \rightarrow T_{USB2} \rightarrow T_{USB3}$$

and so on.

This prevents one device from continuously occupying the bus.

Final Design

USB + Thunderbolt + Priority Interrupts + Software Interrupts + Round – Robin Arbitration

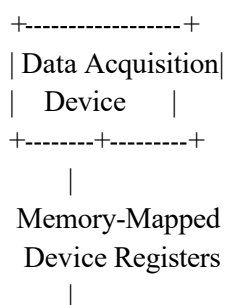
This architecture provides high performance while preventing interrupt and bus starvation.

4. Data Acquisition System at 500 MB/s

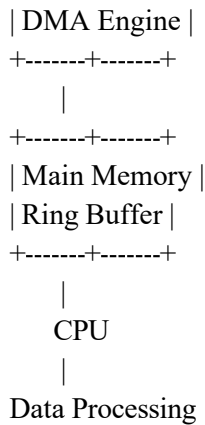
Given Constraints

- Continuous data stream = **500 MB/s**
- Minimal CPU intervention
- Compare DMA with interrupt-driven I/O
- Use memory-mapped I/O for device registers

Proposed Architecture



+-----+-----+



Memory-Mapped Registers

The acquisition device has registers such as:

0x50000000 → Control Register

0x50000004 → Status Register

0x50000008 → Data Register

0x5000000C → DMA Address

0x50000010 → DMA Length

The CPU accesses these registers using normal memory instructions.

DMA Transfer

DMA is the preferred method for a continuous 500 MB/s data stream.

Acquisition Device



DMA



Ring Buffer in RAM

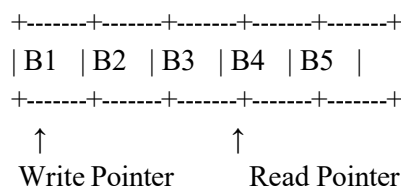


CPU Processing

The CPU only configures the DMA engine and processes completed buffers.

Ring Buffer

A circular buffer is suitable for continuous acquisition.



The acquisition device continuously writes new data while the CPU reads previously completed buffers.

DMA vs Interrupt-Driven I/O

Feature	DMA	Interrupt-Driven I/O
CPU involvement	Very low	High
Suitable for 500 MB/s	Yes	No/poor
Data movement	Hardware	CPU
Interrupt frequency	Low	Potentially very high
CPU overhead	Low	High
Throughput	High	Lower
Power efficiency	Better	Worse
Best use	Large continuous data	Small transfers

Interrupt-Driven Example

Suppose each interrupt transfers 1 KB.

At 500 MB/s:

$$500MB/s \div 1KB$$

Approximately:

$$500,000 \text{ interrupts/s}$$

This creates enormous CPU overhead.

Therefore, interrupt-driven I/O is unsuitable.

DMA Example

Suppose DMA transfers 1 MB per buffer.

$$500MB/s \div 1MB = 500 \text{ transfers/s}$$

Only about 500 completion events per second are required.

Thus, DMA greatly reduces CPU overhead.

Double Buffering

Two buffers can be used:

Buffer A → CPU processing

Buffer B → DMA acquisition

After Buffer B is filled:

Buffer B → CPU

Buffer A → DMA

This allows acquisition and processing to occur simultaneously.

Final Design

Memory – Mapped I/O + DMA + Ring Buffer + Interrupt on Buffer Completion

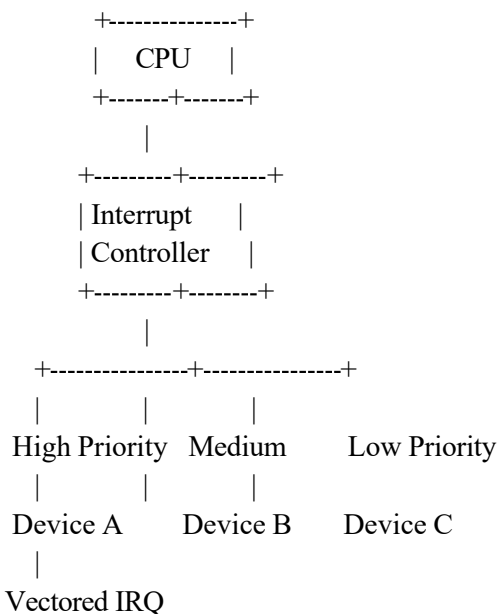
DMA is clearly superior to interrupt-driven I/O for the required **500 MB/s continuous stream**.

5. Nested Interrupt Handling Mechanism

Given Constraints

- Support nested interrupts
- Maximum interrupt latency < **10 μs**
- Vectored interrupts for high-priority devices
- Software interrupts for background tasks

Proposed Architecture



Interrupt Priority

Assign priorities:

Priority 0 → Emergency / critical device

Priority 1 → High-speed device

Priority 2 → Normal device

Priority 3 → Background task

A high-priority interrupt can interrupt a currently executing lower-priority ISR.

Example

Suppose:

CPU executing ISR for Device B



High-priority Device A generates IRQ



CPU saves Device B context



CPU executes ISR-A



ISR-A completes



CPU restores Device B context



CPU continues ISR-B

This is called **nested interrupt handling**.

Context Saving

When a higher-priority interrupt arrives, the processor saves the current context:

Program Counter

Status Register

General Registers

Stack Pointer

The new ISR executes using the saved context.

After completion:

Restore Context



Return From Interrupt



Resume Previous Task

Vectored Interrupts

High-priority devices are assigned dedicated vectors.

For example:

Vector 0x10 → Timer ISR

Vector 0x14 → High-Speed Sensor ISR

Vector 0x18 → Network ISR

Vector 0x1C → Storage ISR

When the interrupt occurs, the CPU directly obtains the ISR address.

This reduces dispatch time.

Meeting the 10 μs Latency Requirement

Interrupt latency can be expressed as:

$$T_L = T_{\text{recognition}} + T_{\text{context}} + T_{\text{vector}} + T_{\text{ISR-start}}$$

The design must satisfy:

$$T_L < 10\mu\text{s}$$

For example:

Operation	Time
Interrupt recognition	1 μ s
Context saving	2 μ s
Vector lookup	0.5 μ s
ISR entry	1 μ s
Total	4.5 μs

Therefore:

$$4.5\mu\text{s} < 10\mu\text{s}$$

The requirement is satisfied.

Short ISR Design

To maintain low latency, the ISR should perform only critical operations:

ISR:

- Save minimum context
- Read device status
- Capture critical data
- Clear interrupt
- Schedule deferred task
- Return

Lengthy operations should not be performed inside the ISR.

Software Interrupts for Background Tasks

Background work can be deferred to a software interrupt:

Hardware Interrupt

↓

Short ISR

↓

Set software-interrupt flag

↓

Software Interrupt

↓

Background Processing

Examples:

- Data formatting
- Logging
- Protocol processing
- Non-critical calculations
- Buffer management

This prevents low-priority processing from increasing the latency of critical hardware interrupts.

Interrupt Masking

Only lower-priority interrupts should be temporarily masked during critical sections.

For example:

High Priority IRQ → Always enabled

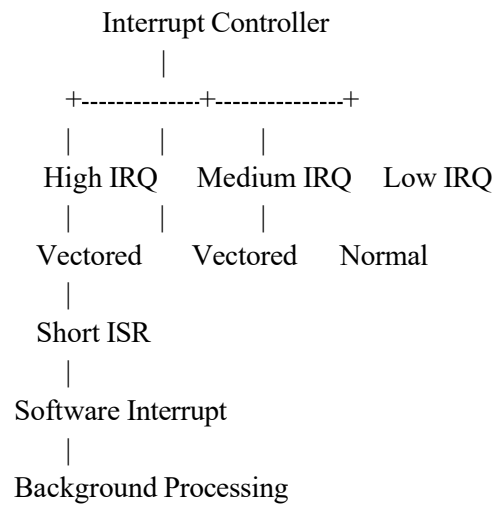
Medium Priority IRQ → Enabled normally

Low Priority IRQ → Temporarily masked

The critical ISR should be very short so that high-priority events are serviced quickly.

Final Architecture

CPU



Final Design

Nested + Vectored Interrupts + Priority Controller + Short ISRs + Software Interrupts

This provides fast response to critical devices while allowing background processing without interfering with real-time interrupt handling.